

The Ultimate Midterm Round-Up

CMPT 211: Introduction to Software Development

Concordia University of Edmonton

Winter 2026

Exam Overview & Objectives

Format: Multiple Choice (Optical Sheet)

Roadmap for this Review:

- ① **Broad Definitions:** Modularity, Scalability, Robustness.
- ② **Paradigms:** Imperative Structure vs. Object-Oriented Programming.
- ③ **Language Architecture:** Compiled vs. Interpreted (PVM).
- ④ **Coding Specifics:** Mutability, Data Structures, and Serialization.
- ⑤ **Functions & Flow:** Signatures and the Call Stack.
- ⑥ **Errors & Handling:** Exception handling (try/except) and Files.

Core Definitions: The "Big Three"

Professional software development relies on these three pillars:

1. Modularity

Breaking a large, complex system down into smaller, independent, and interchangeable components. *Example: A "stack of shallow modular functions" where each does exactly one thing.*

2. Scalability

The capability of a system to handle a growing amount of work gracefully. *Example: Code that processes 10 records just as accurately as 1,000,000 records without needing a rewrite.*

3. Robustness

The ability of a system to cope with errors during execution and erroneous input. A robust program degrades gracefully instead of crashing entirely.

4. Interpretability

The measure of how easily a human reader (like another developer) can understand the logic, structure, and intent of the code without needing an external manual.

Why does it matter?

- Code is read by humans approximately 10 times more often than it is written.
- Clear variable names, modular structures, and focused functions drastically increase interpretability.

Paradigms: Imperative vs. OOP

A "Paradigm" is a fundamental style or logical approach to writing software.

1. Imperative (Procedural) Structure

- **Design:** Data (variables) and behavior (functions) are completely **separate**.
- **Flow:** Code is a sequence of explicit commands. Data is passed manually from one function down to the next.

2. Object-Oriented Programming (OOP)

- **Design: Encapsulation.** Data (attributes) and behavior (methods) are bundled together into logical "Objects" (Classes).
- **Flow:** Objects interact with each other by calling their built-in methods.

Visualizing the Paradigms

Imperative Approach

```
# Data is standalone
grades = [85, 90, 92]

# Function is standalone
def get_average(g_list):
    return sum(g_list)/len(g_list)

# We must pass data into the
  function
avg = get_average(grades)
```

OOP Approach

```
1  # Object holds BOTH data &
    function
2  class Student:
3      def __init__(self, grades):
4          self.grades = grades
5
6      def get_average(self):
7          return sum(self.grades) /
            len(self.grades)
8
9  # The object handles itself
10 alice = Student([85, 90, 92])
11 avg = alice.get_average()
```

Compiled vs. Interpreted Languages

Compiled Languages (e.g., C)

- A compiler translates the *entire* source code into binary **machine code** all at once.
- Creates a hardware-dependent executable.
- **Pro:** Extremely fast.
- **Con:** Not portable across different OS architectures.

Interpreted Languages (Python)

- Translates code into an intermediate step called **bytecode**.
- The **Python Virtual Machine (PVM)** executes this bytecode line-by-line.
- **Pro:** "Write once, run anywhere."
- **Con:** Real-time translation adds slight overhead.

The Foundation: Mutable vs. Immutable

Before choosing a container, ask: "**Are we allowed to change the contents?**"

1. Mutable (The Whiteboard)

- You can add, remove, and rewrite.
- **Examples:** Lists [], Dictionaries {}.
- **Risk:** Accidental changes or side-effects.

2. Immutable (The Stone Tablet)

- Once created, it is locked in memory.
- **Examples:** Tuples (), Strings "", Primitives (int, float).
- **Benefit:** Mathematical safety & Speed.

Lists vs. Tuples (The Code)

Lists: The "Backpack" (Mutable Sequence)

```
grades = [85, 90, 78]

# 1. Parsing (Index)
first = grades[0]

# 2. Modifying (Mutation)
grades.append(95)
grades[0] = 100

# Success!
```

Tuples: The "Sealed Box" (Immutable Sequence)

```
1 location = (53.54, -113.49)
2
3 # 1. Parsing (Index)
4 lat = location[0]
5
6 # 2. Modifying (Mutation)
7 location[0] = 54.00
8 # CRASH! TypeError
9 # Carved in stone!
```

Dictionaries: The Tree Structure

A Dictionary is not just a flat map; it is a **Tree**. It does not use numerical indexes. It maps a **Key** to a **Value**.

```
# Parsing Nested Data (Climbing the Tree)
student_system = {
    "profile": {                # Branch
        "name": "Alice"        # Leaf
    }
}

# Reading left-to-right climbs down the tree
print(student_system["profile"]["name"])
```

Serialization: JSON vs. Python

Think of JSON as **Dehydrated Food**. You must parse it to turn it into memory objects.

1. JSON String (Frozen)

```
import json
# Just a giant text string!
raw = '{"id": 101}'
# raw["id"] -> CRASH!
```

2. Python Dict (Thawed)

```
1 # Parse into memory
2 data = json.loads(raw)
3 # NOW we can use it
4 print(data["id"]) # 101
```

Functions & Signatures

Definition: A reusable, self-contained block of code designed to perform a *single* specific task.

The Signature: The "blueprint" telling the compiler/interpreter how to identify the function and what inputs it expects.

```
# Signature: Name is 'calculate_tax', Parameters are 'amount' & 'rate'
def calculate_tax(amount, rate):
    tax = amount * rate
    return tax
```

The Call Stack (LIFO Memory)

How Python tracks execution across functions:

- Uses a **Last-In, First-Out (LIFO)** structure.
- When `main()` calls `calc_tax()`, `main()` pauses.
- `main()`'s memory address is **pushed** onto the Stack.
- When `calc_tax()` hits `return`, it is **popped** off the Stack.
- Python looks at the top of the Stack, sees `main()`, and resumes.

Exception Handling: The "Inner Shield"

Fragile Engineering: Assuming data is always perfect. A fragile program encounters an error and suffers a fatal crash.

Robust Engineering (try/except): Anticipating failures. The shield catches the error *before* it destroys the execution environment.

```
def calculate_age(input_year):  
    try:  
        # The "Danger Zone"  
        year = int(input_year)  
        return 2026 - year  
    except ValueError:  
        # The "Shield" activates  
        return "Error: Please enter a numeric year."
```

Common Exceptions to Know

You must know *which* shield to deploy. Catching specific errors is better practice than catching everything blindly.

- **ValueError:** Argument has the right type but an inappropriate value (e.g., `int("Hello")`).
- **TypeError:** Operation applied to an inappropriate type (e.g., `"Score: " + 95`).
- **FileNotFoundError:** Trying to read a file that does not exist.
- **ZeroDivisionError:** Second argument of a division operation is zero.

Applying Robustness to File Handling

File I/O is a primary source of crashes. We use robustness to handle it.

```
# 1. Fragile Reading (Will crash if file is missing)
f = open("data.csv", "r")

# 2. Robust Reading ("Inner Shield")
try:
    f = open("data.csv", "r")
except FileNotFoundError:
    print("Warning: File missing. Using default settings.")

# 3. Robust Writing (Mode "a" safely auto-creates missing files!)
with open("results.txt", "a") as f:
    f.write("New data\n")
```

Good luck on the Midterm!

Focus on understanding the *why*, not just the *what*.